

X09 STUDIO

SOFTWARE FACTORY IA

PROJECT BLUEPRINT v2.1

Documento de produto, experiência, arquitetura funcional e engenharia visual para reconstrução do X09 Studio.

REVISÃO CRÍTICA: esta versão preserva o núcleo do Blueprint original e acrescenta o Asset Intelligence Engine, geração visual orientada por finalidade e regras de validação para impedir assets incoerentes.

“O usuário descreve o que quer. O X09 entende, cria, visualiza e evolui o projeto.”

Versão legível • alto contraste • fonte incorporada • pronta para servir como fonte de verdade de implementação.

1. Objetivo e visão do produto

Base preservada do Blueprint v1.0, com expansão específica do sistema visual.

O X09 Studio deve ser uma plataforma premium para criação de software com IA, com experiência pública para usuários e uma base preparada para evolução futura. O usuário não precisa conhecer termos técnicos para iniciar um projeto. Ele descreve a ideia em linguagem natural e acompanha o resultado em um preview funcional.

Visão ampliada v2.1: o Studio não deve agir apenas como um gerador de código. Ele deve funcionar como uma Software Factory: interpretar a intenção, planejar a experiência, definir a identidade visual, determinar quais assets são necessários, buscar ou gerar esses assets, construir o software e validar o resultado antes de apresentá-lo como concluído.

Princípios fundamentais:

- O usuário descreve o que quer construir em linguagem natural.
- O Studio transforma a solicitação em um projeto estruturado.
- O usuário acompanha o resultado em preview sem abandonar o chat.
- O sistema mantém histórico e estado do projeto.
- O design deve parecer produto premium, não painel administrativo genérico.
- A interface para o usuário evita jargão técnico desnecessário.
- Assets visuais são tratados como parte do produto, não como preenchimento de template.

2. Fluxo principal do usuário

Fluxo base: Landing → Cadastro/Login → Dashboard → primeiro comando → Workspace de Desenvolvimento → Chat + Preview → salvar projeto → continuar evoluindo.

Etapa	Comportamento esperado
Landing	Apresenta o X09 Studio como uma experiência premium de criação com IA.
Login/Cadastro	Autenticação simples, social quando configurada, sem expor detalhes técnicos.
Dashboard	Usuário inicia um projeto, escolhe modelo ou retoma projeto existente.
Primeiro comando	O agente interpreta intenção e cria o projeto inicial.
Planejamento visual	O sistema identifica telas, conteúdo e assets necessários.
Workspace	Chat à esquerda; preview executável à direita.
Construção	Agente trabalha em etapas, mostrando estados compreensíveis.
Validação	Build, preview e assets são verificados antes de serem considerados concluídos.
Evolução	Usuário pede alterações no mesmo chat e o projeto mantém estado/versionamento.

3. Direção visual e experiência de interface

A referência aprovada estabelece uma linguagem dark premium, com superfícies profundas, gradientes luminosos, bordas discretas, brilho controlado, tipografia forte, cards modernos e microinterações. A referência é visual: não copiar literalmente todos os elementos.

Regra	Diretriz
Workspace	Desktop first; responsividade obrigatória.
Contraste	Alto contraste e hierarquia clara.
Botões	Destaque luminoso para ação primária; estados hover/active/disabled.
Cards	Profundidade visual sem excesso de efeitos.
Movimento	Transições curtas e suaves; nada deve parecer lento.
Mascote	Robot X09 como elemento de orientação e estado, não como decoração aleatória.
Linguagem	Mensagens amigáveis e não técnicas para o usuário final.

Regra de produto: aparência premium deve coexistir com clareza. Efeito visual nunca deve prejudicar leitura, navegação ou percepção de progresso.

4. Telas obrigatórias

As telas abaixo preservam o núcleo do Blueprint original e incorporam a experiência visual definida para o novo X09 Studio.

4.1 Landing Page — Premium Showcase

Header com logo X09 Studio, Produtos, Recursos, Preços, Blog, Entrar e Começar agora. Hero com headline forte, subtítulo, CTAs, métricas e elemento visual de IA. Recursos: geração com IA, design, apps Web/Mobile, banco, deploy e escalabilidade. CTA final para cadastro. Efeitos: entrada progressiva, glow, parallax sutil e hover.

4.2 Login e Cadastro

Login com e-mail + senha, recuperação e provedores sociais quando configurados. Cadastro com nome, e-mail, senha e aceite. O mascote pode aparecer de forma integrada à composição visual.

4.3 Dashboard inicial

Menu lateral fixo/colapsável. Início, Meus Projetos, Modelos, Biblioteca, Créditos, Integrações, Deploy, Comunidade e Configurações. Área principal com saudação, campo para iniciar projeto, atalhos e projetos recentes. Saldo visível e sem saldo negativo.

4.4 Tela inicial de criação

O usuário encontra um campo de criação central e opções de modelos prontos. O mascote aparece de forma proeminente e pode interagir visualmente com a área de comando. O objetivo é fazer a pessoa entender imediatamente: “eu descrevo e o X09 constrói”.

4.5 Workspace de Desenvolvimento

Chat à esquerda; Preview à direita. Chat mantém conversa, solicitações, estados das tarefas e novos comandos. Preview executável mostra o projeto real. Abas/ações: Preview, Código, Arquivos e Deploy.

4.6 Biblioteca e assets

Área para visualizar assets usados no projeto, identificar origem, finalidade, status de aprovação e permitir substituição quando apropriado.

4.7 Projetos

Lista de projetos com nome, tipo, última atualização, estado e ação de continuar. O projeto deve preservar contexto e versões.

5. Mascote e estados de trabalho da IA

Durante a execução, o usuário não deve ser exposto a logs técnicos, stack traces, nomes de ferramentas, chamadas internas ou detalhes de infraestrutura. O sistema comunica progresso por meio do mascote e de estados humanos.

Estado	Exemplo de mensagem ao usuário
Pensando	“Estou entendendo sua ideia e definindo a melhor estrutura.”
Analisando	“Estou analisando o que seu projeto precisa.”
Planejando	“Estou organizando as páginas e funcionalidades.”
Criando	“Estou construindo sua primeira versão.”
Criando imagens	“Estou preparando imagens específicas para o seu projeto.”
Buscando referências	“Estou procurando elementos visuais adequados.”
Executando	“Estou colocando tudo para funcionar.”
Revisando	“Estou revisando o resultado antes de entregar.”
Ajustando	“Encontrei um detalhe que precisa de ajuste.”
Concluído	“Pronto. Sua versão está disponível para você testar.”

Regra: estados são parte da experiência, não uma reprodução de logs internos. Se houver erro, a interface explica o que o usuário pode fazer, enquanto os detalhes técnicos ficam disponíveis apenas onde fizer sentido para desenvolvimento/diagnóstico.

6. NOVO PILAR — Asset Intelligence Engine

Esta é a principal expansão da versão v2.1.

O X09 não deve tratar imagens como URLs aleatórias adicionadas ao código. Antes de construir a interface, a IA deve identificar quais assets a experiência exige e definir a finalidade de cada um.

Exemplo: o usuário escreve “Quero um site para minha loja de roupas”. O Studio pode identificar hero, banners, categorias, produtos, acessórios e imagens de apoio. Para cada necessidade, ele decide se deve usar um asset enviado pelo usuário, buscar uma imagem real ou gerar uma imagem específica com IA.

Regra central: o LLM pode solicitar um asset, mas não pode fabricar uma URL de asset.

Fonte	Quando usar
Asset do usuário	Logo, fotos reais da loja, produtos reais ou qualquer material fornecido pelo usuário.
Banco de imagens	Quando uma fotografia real existente atende melhor ao objetivo e aos requisitos de licença/origem.
Geração por IA	Quando a composição precisa ser criada especificamente para o layout, conceito ou identidade do p
Fallback	Se a fonte escolhida não produzir um asset válido, o sistema tenta outra estratégia.

7. Visual Brief automático por projeto

Antes de buscar ou gerar imagens, o Studio deve construir internamente um Visual Brief. Esse documento orienta tanto o Asset Engine quanto o gerador do projeto.

Exemplo de estrutura:

```
business: loja de roupas femininas
style: premium / editorial / moderno
palette: definida pela identidade do projeto
audience: público feminino
hero: modelo + coleção + espaço negativo para texto
sections: nova coleção, categorias, produtos, benefícios, CTA
asset_policy: preferir assets do usuário; gerar imagens conceituais quando necessário
```

Cada asset deve possuir uma especificação de finalidade:

- tipo: imagem, ícone, ilustração, vídeo ou outro asset suportado
- uso: hero, card, banner, categoria, produto, fundo etc.
- assunto: objeto ou cena que precisa aparecer
- estilo: editorial, comercial, minimalista, tecnológico etc.
- composição: posição do sujeito, espaço para texto e enquadramento
- orientação e proporção: horizontal, vertical, quadrada ou outra
- qualidade mínima e resolução necessária
- restrições: não inserir objetos incompatíveis com o negócio ou com o contexto
- status: solicitado, gerando, candidato, aprovado, rejeitado ou substituído

8. Geração de imagem orientada pela finalidade

A geração não deve receber apenas “gere uma imagem bonita”. O prompt interno precisa carregar o contexto do local onde a imagem será usada.

Exemplo — Hero de loja de roupas:

Purpose: hero da homepage
Subject: modelo feminina usando coleção contemporânea
Environment: boutique de moda sofisticada
Composition: modelo posicionada à direita, espaço negativo à esquerda
Lighting: fotografia editorial de moda
Palette: coerente com a identidade visual
Format: 16:9
Constraint: imagem criada para receber headline e CTA no lado esquerdo

A mesma lógica muda conforme a finalidade. Uma imagem para card de produto não deve ser gerada com a mesma composição de um hero. Um banner promocional possui outras necessidades. Uma imagem de fundo possui outras restrições.

Objetivo: cada imagem deve parecer ter sido criada para aquele projeto e para aquele espaço específico da interface.

9. Busca em fontes externas — sem URLs inventadas

Quando o Studio decidir buscar uma imagem existente, o Asset Engine deve consultar provedores por meio de integrações reais. A resposta do provedor deve fornecer os dados necessários para identificar a mídia e sua origem. O LLM não escolhe uma URL “no chute”.

Provedor	Papel no engine
Pexels	Busca de fotografias e vídeos conforme parâmetros disponíveis na API.
Unsplash	Busca de fotografias com filtros e regras próprias de uso da API.
Pixabay	Busca de imagens conforme filtros da API.
Openverse	Busca de mídia com licenças abertas; licença individual deve ser verificada.

As regras de uso, atribuição, hotlink/download e limites de cada provedor devem ser implementadas conforme a documentação oficial vigente no momento da integração.

Referências oficiais consultadas para esta versão: [Pexels API Documentation](#); [Unsplash API Documentation](#) e [API Guidelines](#); [Pixabay API Documentation](#); [Openverse Documentation](#).

10. Validação visual — obrigatório

Encontrar uma imagem não significa aprová-la. O Asset Engine precisa validar se o resultado corresponde ao pedido.

Validação	Pergunta
Semântica	A imagem representa realmente o assunto solicitado?
Contexto	A imagem combina com o tipo de negócio e seção?
Composição	Existe espaço e enquadramento adequados para o layout?
Formato	A proporção e orientação são adequadas?
Qualidade	A resolução atende ao uso?
Coerência	A imagem combina com as demais imagens do projeto?
Origem/licença	O sistema possui origem e metadados necessários?
Preview	Depois de inserida, a imagem funciona visualmente na interface?

Se a validação falhar, o asset deve ser rejeitado. O sistema busca outro candidato ou gera novamente. Não deve aceitar uma imagem errada apenas para “preencher o espaço”.

11. Fluxo completo do Asset Intelligence Engine

Fluxo de referência para implementação:

```
Usuário descreve o projeto
↓
AI Orchestrator entende negócio, público e objetivo
↓
Project Engine identifica páginas e componentes
↓
Visual Brief define identidade e necessidades visuais
↓
Asset Engine cria Asset Requests específicos
↓
Decisão: usuário / busca externa / geração IA
↓
Candidatos são obtidos ou gerados
↓
Validação semântica + técnica + visual
↓
Asset aprovado é registrado com metadados
↓
Project Engine insere o asset no projeto
↓
Preview executa a interface
↓
Quality Check verifica resultado final
↓
Se necessário: rejeitar → substituir/regenerar → validar novamente
```

Princípio: o ciclo de validação termina no preview, não na simples geração do arquivo.

12. Problemas que esta arquitetura deve impedir

Falha antiga	Regra v2.1
Carro no lugar de janela	Asset Request + validação semântica e contextual antes de aprovação.
Personagem 3D no lugar de comida	Categoria, assunto e finalidade obrigatórios; asset incompatível é rejeitado.
Imagem bonita, mas inadequada ao contexto	Validação de composição e teste no preview.
URL aleatória inventada pela IA	Somente URLs/metadados retornados por integração ou assets internos aprovados.
Mesmo estilo de imagem para tudo	Cada asset possui finalidade e composição próprias.
Site genérico	Visual Brief + direção visual + componentes adequados ao negócio.
Template rígido	IA decide estrutura e assets com base na intenção do usuário.

13. Arquitetura funcional

Camada	Responsabilidade
Frontend	Landing, Auth, Dashboard, Workspace, componentes e responsividade.
API/Backend	Usuários, projetos, mensagens, tarefas, créditos, arquivos, execução e integrações.
Banco	Perfis, projetos, versões, mensagens, tarefas, créditos, consumo e configurações.
AI Orchestrator	Recebe intenção, escolhe fluxo/modelo, registra execução e retorna resultado.
Project Engine	Converte comandos em tarefas e mantém estado do projeto.
Asset Intelligence Engine	Planeja, busca, gera, valida, registra e entrega assets.
Preview Engine	Gera/atualiza ambiente de visualização do projeto.
Quality Engine	Executa verificações de build, experiência e coerência visual.
Storage	Arquivos, assets, imagens, exports e artefatos.

A implementação deve ser modular. O Studio não deve acoplar a interface diretamente a um provedor específico de IA ou de assets. O acesso a modelos, geração de imagens e provedores de mídia deve passar por camadas de orquestração.

14. Modelo de dados inicial — expansão v2.1

Entidade	Objetivo principal
users	Identidade e dados básicos.
profiles	Preferências, avatar, plano e configurações.
projects	Nome, descrição, owner, status, stack e timestamps.
project_versions	Versões do projeto, hash/referência e estado de build.
chat_threads / chat_messages	Conversas por projeto.
tasks	Tarefas geradas pelo Project Engine e seus estados.
credits_ledger	Entradas, débitos, estornos e saldo auditável.
ai_usage	Modelo/provedor, operação, unidades consumidas e custo interno.
files/assets	Arquivos, imagens e artefatos associados.
asset_requests	Necessidades visuais, finalidade, especificação e status.
asset_candidates	Resultados obtidos de provedores ou gerações.
asset_validations	Resultados das verificações semânticas, técnicas e visuais.
visual_briefs	Identidade visual e mapa de necessidades do projeto.
deployments	Versão, ambiente, status e logs essenciais.

15. Créditos e IA

O sistema de créditos deve ser centralizado. Nenhuma função de IA pode executar uma operação cobrada sem registrar previamente a intenção de consumo e validar o saldo.

Fluxo:

Solicitação → identificar operação → estimar custo → validar créditos → reservar/débito → executar → registrar consumo → finalizar/estornar quando aplicável.

- Nunca permitir saldo negativo.
- Registrar cada consumo em ledger auditável.
- Separar créditos promocionais de créditos comprados, se aplicável.
- Permitir limites por usuário/projeto/operação.
- Provedor e modelo configuráveis no backend.
- Operações de geração de imagem devem possuir registro próprio quando consumirem recursos cobrados.

16. Ordem de implementação — objetiva e iterativa

A implementação deve seguir uma sequência objetiva e testável. Evitar construir módulos futuros antes de validar o fluxo principal.

- Preparar projeto, ambiente e estrutura base.
- Implementar Landing Page e sistema visual.
- Implementar autenticação e persistência do usuário.
- Implementar Dashboard e navegação.
- Implementar chat inicial e criação de projeto.
- Implementar Workspace com Chat + Preview.
- Implementar Project Engine básico e tarefas.
- Implementar Asset Intelligence Engine mínimo: Asset Request → fonte → validação → aprovação.
- Implementar geração de imagem orientada por finalidade.
- Implementar salvamento/versionamento de projeto e assets.
- Implementar créditos e registro de consumo.
- Implementar tratamento de erros e estados de build.
- Implementar Quality Check do preview.
- Testar fluxo completo de ponta a ponta.
- Somente depois avaliar próximos módulos.

Método de execução: concluir a etapa atual, testar o resultado e somente então definir a próxima mudança. Não antecipar módulos desnecessários.

17. Quality Engine — critério de “pronto”

O X09 não deve considerar um projeto pronto apenas porque o código compilou. A qualidade precisa ser observada em camadas.

Camada	Verificação
Build	Projeto compila sem erro bloqueante.
Runtime	Aplicação inicia e navega pelas rotas previstas.
UX	CTAs, formulários e interações principais funcionam.
Responsividade	Desktop e mobile possuem layout utilizável.
Visual	Hierarquia, espaçamento, contraste e composição são coerentes.
Assets	Imagens correspondem ao contexto e finalidade.
Preview	Resultado executável representa a última versão válida.
Persistência	Projeto, mensagens e versão permanecem após recarregar.

18. Instruções para o agente de desenvolvimento

Este Blueprint deve ser utilizado como fonte de verdade da implementação. O agente deve evitar inventar funcionalidades fora do escopo principal enquanto o fluxo central não estiver funcionando.

- Implementar por etapas pequenas.
- Após cada etapa, validar build e comportamento.
- Não apagar funcionalidades existentes sem verificar dependências.
- Não colocar chaves secretas no frontend ou no repositório.
- Manter componentes reutilizáveis.
- Priorizar experiência e estabilidade do fluxo principal.
- Quando houver decisão técnica não especificada, escolher a alternativa mais simples que preserve a arquitetura.
- Registrar mudanças relevantes em documentação/changelog.
- Não fabricar URLs de assets.
- Não aprovar assets sem validação.
- Não expor logs técnicos ao usuário final.
- Concluir a etapa atual antes de planejar a próxima.

Importante: se uma imagem necessária não puder ser obtida ou gerada com qualidade suficiente, o agente deve sinalizar a necessidade e escolher uma estratégia de substituição válida; não deve inserir um asset incompatível apenas para preencher o componente.

19. Checklist de aceite v2.1

- ☐ Landing responsiva e visualmente consistente com a direção aprovada.
- ☐ CTAs e navegação funcionando.
- ☐ Login com e-mail/senha funcionando.
- ☐ Cadastro funcionando.
- ☐ Google/Facebook OAuth preparado/funcionando conforme configuração.
- ☐ Dashboard protegido por autenticação.
- ☐ Menu lateral funcionando e responsivo.
- ☐ Saldo de créditos visível e consistente.
- ☐ Primeiro comando cria projeto.
- ☐ Primeiro comando abre o Workspace automaticamente.
- ☐ Chat permanece disponível durante o desenvolvimento.
- ☐ Preview é atualizado após execução bem-sucedida.
- ☐ Erros de execução são apresentados de forma compreensível.
- ☐ Projeto e mensagens persistem após recarregar.
- ☐ Nenhum segredo fica exposto no frontend.
- ☐ Visual Brief é criado para novos projetos.
- ☐ Asset Requests são gerados por finalidade.
- ☐ LLM não fabrica URLs de assets.
- ☐ Assets podem vir do usuário, de provedores ou de geração IA.
- ☐ Assets passam por validação antes de aprovação.
- ☐ Assets rejeitados podem ser substituídos/regenerados.
- ☐ Imagens são geradas considerando o local de uso no layout.
- ☐ Preview final passa por Quality Check.
- ☐ Fluxo principal testado em desktop e mobile.

20. Instrução-mestra para o agente

Use o documento inteiro como fonte de verdade. Construa o X09 Studio como uma Software Factory orientada à experiência. O usuário fornece intenção; o sistema interpreta, planeja, cria, visualiza, valida e permite evoluir o projeto. Preserve o fluxo central e implemente iterativamente.

Prioridades absolutas:

- Experiência premium e clara.
- Workspace Chat + Preview como núcleo.
- Mascote X09 como elemento de estado e orientação.
- Zero jargão técnico desnecessário na experiência do usuário.
- Asset Intelligence como parte do processo de construção.
- Geração de imagens orientada pela finalidade de cada espaço.
- Nenhuma URL de asset inventada.
- Validação antes da aprovação e validação novamente no preview.
- Persistência e versionamento.
- Arquitetura modular e independente de provedor.
- Execução iterativa: fazer → testar → analisar → próxima etapa.

Fora do escopo desta versão: preços de venda, margens, custos comerciais, estratégia de negociação e informações destinadas exclusivamente à X09 Software House.

X09 STUDIO

PROJECT BLUEPRINT v2.1 — FINAL

Este documento consolida o Blueprint original e a nova camada de inteligência visual. O objetivo não é apenas gerar software funcional, mas fazer o usuário perceber o X09 Studio como uma ferramenta capaz de transformar uma ideia em um produto visualmente coerente, funcional e evolutivo.

FIM DO DOCUMENTO